SipLog

DA219B Fullstack Lab Report
Student: Ayah Assaad Liljekvist

1. System Overview

SipLog is a full stack wine tasting journal designed for people who want to keep detailed and personal records of the wines they try. The application allows users to create wine entries, attach tasting notes, upload photos, rate wines, and mark bottles they would buy again. Unlike a simple notes app, SipLog stores structured tasting information such as appearance, nose, palate, acidity, body, tannin, mood tags, and personal reflections. The app is intended for wine lovers who want a searchable and visual way to remember their tasting experiences. The project uses React with Vite for the frontend, Express for the backend API, and MongoDB Atlas for cloud database storage. The main problem the app solves is helping users organise and revisit wine tasting experiences in a structured digital format.

2. Database Design

SipLog uses three MongoDB collections, which are users, wines and tastings.

Users

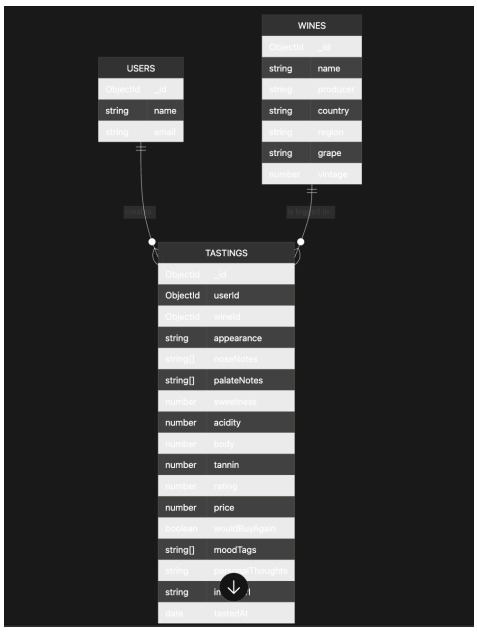
The users collection stores the people who create tasting entries and each user has a name and email. This collection exists so tasting notes can be linked to a specific person.

Wines

The wines collection stores information about the wine itself. Each wine has a name, producer, country, region, grape and its vintage (year). This makes the structure more efficient because the same wine can be referenced by multiple tastings without repeating all wine information every time.

Tastings

The tastings collection is the main entity of the application. It stores the tasting experience for a specific wine, here each tasting includes a userId, wineId, appearance, noseNotes, palateNotes, sweetness, acidity, body, tannin, rating, price, wouldBuyAgain, moodTags, personalThoughts, imageUrl and tastedAt. This collection is the most important part of the project because it reflects what matters in a real wine tasting journal. Here is the ERD:



3. Two API Endpoints

Endpoint 1: Create a Tasting
Route: POST /api/tastings
Expected Request Body:

```json
{
  "userId": "69e681b2b977676391c086f2",
  "wineId": "69e6893fb977676391c086fc",
  "appearance": "Deep ruby",
  "noseNotes": ["black cherry", "cedar", "vanilla"],
  "palateNotes": ["plum", "spice", "oak"],
  "sweetness": 1,
  "acidity": 4,
  "body": 4,
  "tannin": 4,
  "rating": 5,
  "price": 229,
  "wouldBuyAgain": true,
  "moodTags": ["Date night", "Dinner party"],
  "personalThoughts": "Silky and polished with enough structure to feel special."
}
```

Response:

```json
{
  "_id": "680b9f00123456789abcdeff",
  "userId": "69e681b2b977676391c086f2",
  "wineId": "69e6893fb977676391c086fc",
  "appearance": "Deep ruby",
  "noseNotes": ["black cherry", "cedar", "vanilla"],
  "palateNotes": ["plum", "spice", "oak"],
  "sweetness": 1,
  "acidity": 4,
  "body": 4,
  "tannin": 4,
  "rating": 5,
  "price": 229,
  "wouldBuyAgain": true,
  "moodTags": ["Date night", "Dinner party"],
  "personalThoughts": "Silky and polished with enough structure to feel special.",
  "imageUrl": "",
  "tastedAt": "2026-04-26T12:30:00.000Z",
  "createdAt": "2026-04-26T12:30:00.000Z",
  "updatedAt": "2026-04-26T12:30:00.000Z",
  "__v": 0
}
```

Endpoint 2: Tasting Statistics
Route: GET /api/tastings/stats/summary
Expected Request Body:
No request body is required.
Response:

```json
{
  "totalTastings": 5,
  "averageRating": 4.0,
  "averagePrice": 233.6,
  "favoritesCount": 3,
  "topMoodTags": [
    { "tag": "Dinner party", "count": 2 },
    { "tag": "Weeknight", "count": 2 },
    { "tag": "Date night", "count": 1 }
  ],
  "topGrapes": [
    { "grape": "Tempranillo", "count": 1, "averageRating": 5.0 },
    { "grape": "Pinot Noir", "count": 1, "averageRating": 4.0 },
    { "grape": "Syrah", "count": 1, "averageRating": 4.0 }
  ]
}
```

4. Reflection

One challenge I faced during development was handling relationships between MongoDB collections correctly, especially when connecting tasting entries to both users and wines. At one point, my backend was returning incomplete wine data because the populate() calls in the controller did not match the correct schema field names. I solved this by reviewing the Mongoose schema carefully, updating the controller to use the correct reference fields (userId and wineId), and checking that the related documents actually existed before saving new tastings. I also improved the validation logic in the controller so the backend does not trust incoming data automatically. This challenge taught me that in a full-stack application, bugs often come from the connection between the frontend, backend, model, and database rather than from only one file. It also helped me better understand how ObjectId references and Mongoose population work in practice.

5. Feature Iteration

One feature that changed significantly during development was the frontend structure for displaying and managing tastings. In an earlier version of the project, most of the frontend logic was placed inside a single App.jsx file. This version worked, but it made the code harder to maintain and harder to explain. That stage of development is represented by commit 253b805 (feat: build sipLog wine tasting app), where the main application functionality existed but the frontend structure was still too large and centralised. Later in development, I refactored the frontend into smaller, clearer components such as tastingForm, filterBar, favoritesShelf, tastingTimeline and heroPanel. This change is shown in commit 75424d7 (refactor: split App.jsx into smaller components and add seminar prep updates).